Building an Integer ALU
Step 2: Separate Four-Bit ALU Functions

Zion
Anamol Khadka, Prashant Subedi, Davos A De Hoyos
CS3339

October 18, 2026

The course-provided LaTeX template was not supplied; this report uses a standalone article layout.

Contents

1 Introduction and goals **3**

2 Interfaces and design choices **3**
2.1 Logic truth table (applied independently to each bit) . . . 3
2.2 Arithmetic and shift boundary cases . . . 3

3 Validation method and measured results **3**

4 Bitwise AND **5**
4.1 Functional circuit diagram . . . 5
4.2 Verilog implementation . . . 5
4.3 Independent unit test . . . 5
4.4 Observed waveform . . . 6

5 Bitwise NAND **7**
5.1 Functional circuit diagram . . . 7
5.2 Verilog implementation . . . 7
5.3 Independent unit test . . . 7
5.4 Observed waveform . . . 8

6 Bitwise OR **9**
6.1 Functional circuit diagram . . . 9
6.2 Verilog implementation . . . 9
6.3 Independent unit test . . . 9
6.4 Observed waveform . . . 10

7 Bitwise NOR **11**
7.1 Functional circuit diagram . . . 11
7.2 Verilog implementation . . . 11
7.3 Independent unit test . . . 11
7.4 Observed waveform . . . 12

8 Bitwise XOR **13**
8.1 Functional circuit diagram . . . 13
8.2 Verilog implementation . . . 13
8.3 Independent unit test . . . 13
8.4 Observed waveform . . . 14

9 Bitwise XNOR **15**
9.1 Functional circuit diagram . . . 15
9.2 Verilog implementation . . . 15
9.3 Independent unit test . . . 15
9.4 Observed waveform . . . 16

10 Bitwise NOT **17**
10.1 Functional circuit diagram . . . 17
10.2 Verilog implementation . . . 17
10.3 Independent unit test . . . 17
10.4 Observed waveform . . . 17

11 Programmable arithmetic shifter **19**
11.1 Functional circuit diagram . . . 19
11.2 Verilog implementation . . . 19
11.3 Independent unit test . . . 19
11.4 Observed waveform . . . 20

12 Unsigned addition with carry **21**
12.1 Functional circuit diagram . . . 21
12.2 Verilog implementation . . . 21
12.3 Independent unit test . . . 21
12.4 Observed waveform . . . 22

13 Unsigned subtraction with carry/no-borrow **23**
13.1 Functional circuit diagram . . . 23
13.2 Verilog implementation . . . 23
13.3 Independent unit test . . . 23
13.4 Observed waveform . . . 24

14 Unsigned multiplication **25**
14.1 Functional circuit diagram . . . 25
14.2 Verilog implementation . . . 25
14.3 Independent unit test . . . 25
14.4 Observed waveform . . . 26

15 Unsigned division and remainder **27**
15.1 Functional circuit diagram . . . 27
15.2 Verilog implementation . . . 27
15.3 Independent unit test . . . 27
15.4 Observed waveform . . . 28

16 Reproducing the results **29**

17 Conclusion **29**

1 Introduction and goals

Step 2 develops and validates twelve independent combinational ALU modules: seven bitwise logic functions, a programmable shifter, and four integer arithmetic operations. The goals are to preserve every specified result bit, make control and boundary cases explicit, and demonstrate correctness through independent exhaustive unit tests and simulation waveforms. No final-stage opcode selector or integrated ALU is included; integration belongs to the final project step.

2 Interfaces and design choices

All data ports are four bits. Inputs A and B are unsigned integers 0–15; X is the primary output. The handout does not specify signed multiplication or division, so this implementation uses unsigned arithmetic. Y carries the high product nibble, division remainder, or shifted-out bits. Only addition and subtraction expose carry-in and carry-out: the handout does not define carry semantics for multiply or divide, whose complete results are instead retained in X and Y. Confirm this interpretation with the instructor if a uniform carry interface is required.

Each function has its own RTL file and dedicated unit-test file. Modules are combinational and require no clock or reset. Delay statements appear only in testbenches. The shift implementation follows the custom fill and overflow rules in the assignment, including the right-shift example's left-aligned overflow.

2.1 Logic truth table (applied independently to each bit)

A	B	AND	NAND	OR	NOR	XOR	XNOR	NOT A
0	0	0	1	0	1	0	1	1
0	1	0	1	1	0	1	0	1
1	0	0	1	1	0	1	0	0
1	1	1	0	1	0	0	1	0

2.2 Arithmetic and shift boundary cases

Function	A	B	Carry-in	X	Other outputs
Add	15	15	1	15	Carry-out=1
Subtract	0	1	1	15	Carry-out=0 (borrow)
Subtract	0	0	0	15	Carry-out=0 (incoming borrow)
Multiply	15	15	–	1	Y=14
Divide	15	4	–	3	Y=3, error=0
Divide	11	0	–	15	Y=11, error=1
Left shift	11	6	–	8	Y=5 (handout example)
Right shift	5	13	–	13	Y=4 (handout example)

3 Validation method and measured results

Icarus Verilog 12.0 compiled each unit separately with SystemVerilog support and warnings enabled. The testbenches enumerate all binary inputs, wait 10 ns after each stimulus, and use case-inequality comparisons so unknown outputs fail instead of being silently accepted. A failed assertion calls `$fatal`; the shell runner preserves a failing exit status through its logging pipeline. VCD traces are captured directly from each device under test, and the plotted figures are cropped from those traces, not fabricated expected waveforms.

Each binary logic operation has 256 cases; NOT has 16; shift has 256; addition and subtraction have 512 each; multiplication and division have 256 each. **All 3,344 cases passed across twelve separate**

unit tests. For the binary gates the oracle evaluates a per-bit truth table. Arithmetic reference models use integer arithmetic, repeated addition, or repeated subtraction. Shift expectations use the closed-form equations above rather than repeating the RTL shift loop.

The exhaustive runs cover zero and maximum operands, every carry input, every shift direction/distance/fill combination, overflowing additions and products, subtraction underflow, exact and inexact division, and every zero divisor. These are functional RTL simulations; they do not establish timing closure, gate counts, physical implementation, or behavior for unknown input states. Larger optional ALU widths are outside this required four-bit submission.

4 Bitwise AND

$X_i = A_i \wedge B_i$. A bit is one only when both inputs are one.

4.1 Functional circuit diagram



This is a functional port diagram, not a synthesized gate-level schematic.

4.2 Verilog implementation

Algorithm 1: Bitwise AND: module

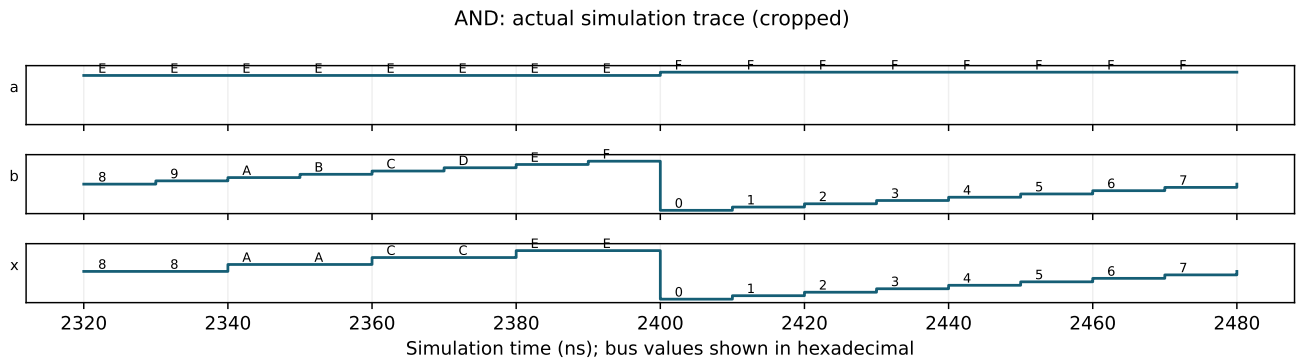
```
‘timescale 1ns/1ps
module alu_and(input wire [3:0] a, input wire [3:0] b, output wire [3:0] x);
  assign x = a & b;
endmodule
```

4.3 Independent unit test

Algorithm 2: Bitwise AND: exhaustive unit test

```
‘timescale 1ns/1ps
module tb_and;
  reg [3:0] a, b;
  reg cin;
  wire [3:0] x;
  integer ia, ib, ic, k, expected, remainder, distance, fill, overflow, count;
  alu_and dut(.a(a), .b(b), .x(x));
  initial begin
    $dumpfile("build/and.vcd");
    $dumpvars(0, dut);
    count=0;
    for (ia=0; ia<16; ia=ia+1)
      for (ib=0; ib<16; ib=ib+1)
        for (ic=0; ic<1; ic=ic+1) begin
          a=ia; b=ib; cin=ic; #10;
          expected = 0;
          for (k=0; k<4; k=k+1)
            expected = expected | (((8 >> (2*((ia>>k)&1)+((ib>>k)&1))) & 1) << k);
          if (x !== expected[3:0]) $fatal(1,"and a=%d b=%d",ia,ib);
          count=count+1;
        end
    $display("PASS and: %0d cases",count);
    $finish;
  end
endmodule
```

4.4 Observed waveform



The crop sweeps B while A=E. Each displayed output follows the per-bit truth table; the testbench checks the full input space beyond this crop.

5 Bitwise NAND

$X_i = \neg(A_i \wedge B_i)$. This complements AND independently for each bit.

5.1 Functional circuit diagram



This is a functional port diagram, not a synthesized gate-level schematic.

5.2 Verilog implementation

Algorithm 3: Bitwise NAND: module

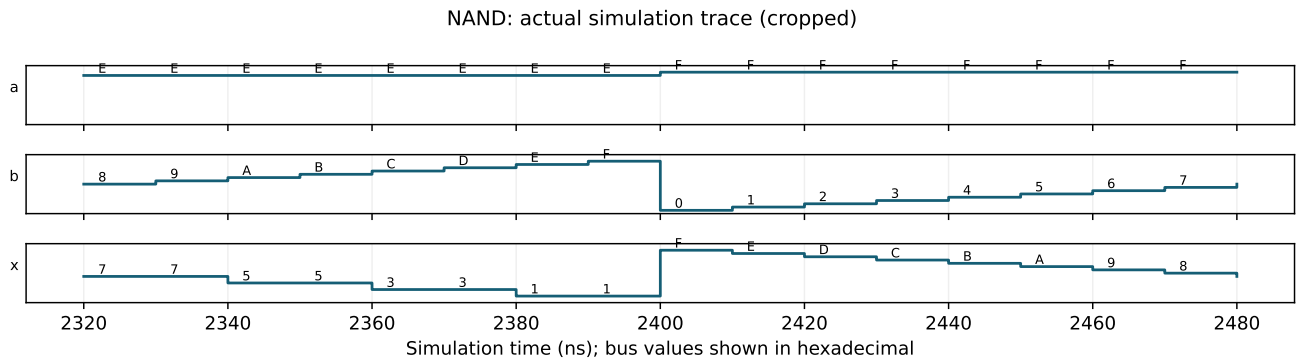
```
‘timescale 1ns/1ps
module alu_nand(input wire [3:0] a, input wire [3:0] b, output wire [3:0] x);
  assign x = ~(a & b);
endmodule
```

5.3 Independent unit test

Algorithm 4: Bitwise NAND: exhaustive unit test

```
‘timescale 1ns/1ps
module tb_nand;
  reg [3:0] a, b;
  reg cin;
  wire [3:0] x;
  integer ia, ib, ic, k, expected, remainder, distance, fill, overflow, count;
  alu_nand dut(.a(a), .b(b), .x(x));
  initial begin
    $dumpfile("build/nand.vcd");
    $dumpvars(0, dut);
    count=0;
    for (ia=0; ia<16; ia=ia+1)
      for (ib=0; ib<16; ib=ib+1)
        for (ic=0; ic<1; ic=ic+1) begin
          a=ia; b=ib; cin=ic; #10;
          expected = 0;
          for (k=0; k<4; k=k+1)
            expected = expected | (((7 >> (2*((ia>>k)&1)+((ib>>k)&1))) & 1) << k);
          if (x !== expected[3:0]) $fatal(1,"nand a=%d b=%d",ia,ib);
          count=count+1;
        end
    $display("PASS nand: %0d cases",count);
    $finish;
  end
endmodule
```

5.4 Observed waveform



The crop sweeps B while A=E. Each displayed output follows the per-bit truth table; the testbench checks the full input space beyond this crop.

6 Bitwise OR

$X_i = A_i \vee B_i$. A bit is one when at least one input bit is one.

6.1 Functional circuit diagram



This is a functional port diagram, not a synthesized gate-level schematic.

6.2 Verilog implementation

Algorithm 5: Bitwise OR: module

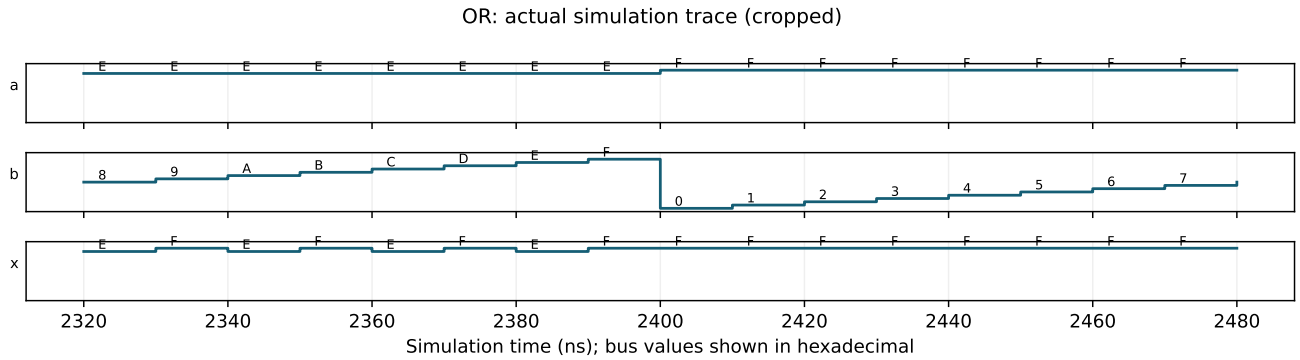
```
‘timescale 1ns/1ps
module alu_or(input wire [3:0] a, input wire [3:0] b, output wire [3:0] x);
  assign x = a | b;
endmodule
```

6.3 Independent unit test

Algorithm 6: Bitwise OR: exhaustive unit test

```
‘timescale 1ns/1ps
module tb_or;
  reg [3:0] a, b;
  reg cin;
  wire [3:0] x;
  integer ia, ib, ic, k, expected, remainder, distance, fill, overflow, count;
  alu_or dut(.a(a), .b(b), .x(x));
  initial begin
    $dumpfile("build/or.vcd");
    $dumpvars(0, dut);
    count=0;
    for (ia=0; ia<16; ia=ia+1)
      for (ib=0; ib<16; ib=ib+1)
        for (ic=0; ic<1; ic=ic+1) begin
          a=ia; b=ib; cin=ic; #10;
          expected = 0;
          for (k=0; k<4; k=k+1)
            expected = expected | (((14 >> (2*((ia>>k)&1)+((ib>>k)&1))) & 1) << k);
          if (x !== expected[3:0]) $fatal(1,"or a=%d b=%d",ia,ib);
          count=count+1;
        end
    $display("PASS or: %0d cases",count);
    $finish;
  end
endmodule
```

6.4 Observed waveform



The crop sweeps B while A=E. Each displayed output follows the per-bit truth table; the testbench checks the full input space beyond this crop.

7 Bitwise NOR

$X_i = \neg(A_i \vee B_i)$. A bit is one only when both input bits are zero.

7.1 Functional circuit diagram



This is a functional port diagram, not a synthesized gate-level schematic.

7.2 Verilog implementation

Algorithm 7: Bitwise NOR: module

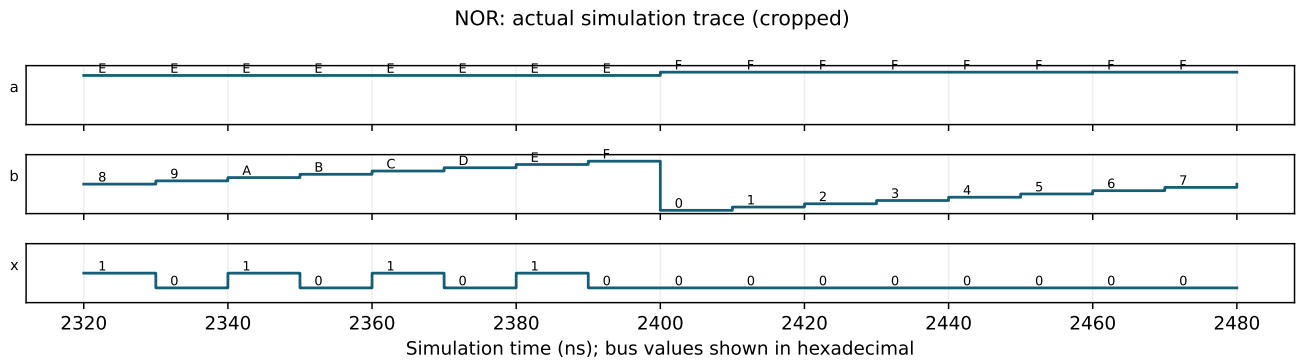
```
`timescale 1ns/1ps
module alu_nor(input wire [3:0] a, input wire [3:0] b, output wire [3:0] x);
  assign x = ~(a | b);
endmodule
```

7.3 Independent unit test

Algorithm 8: Bitwise NOR: exhaustive unit test

```
`timescale 1ns/1ps
module tb_nor;
  reg [3:0] a, b;
  reg cin;
  wire [3:0] x;
  integer ia, ib, ic, k, expected, remainder, distance, fill, overflow, count;
  alu_nor dut(.a(a), .b(b), .x(x));
  initial begin
    $dumpfile("build/nor.vcd");
    $dumpvars(0, dut);
    count=0;
    for (ia=0; ia<16; ia=ia+1)
      for (ib=0; ib<16; ib=ib+1)
        for (ic=0; ic<1; ic=ic+1) begin
          a=ia; b=ib; cin=ic; #10;
          expected = 0;
          for (k=0; k<4; k=k+1)
            expected = expected | (((1 >> (2*((ia>>k)&1)+((ib>>k)&1))) & 1) << k);
          if (x !== expected[3:0]) $fatal(1,"nor a=%d b=%d",ia,ib);
          count=count+1;
        end
    $display("PASS nor: %0d cases",count);
    $finish;
  end
endmodule
```

7.4 Observed waveform



The crop sweeps B while A=E. Each displayed output follows the per-bit truth table; the testbench checks the full input space beyond this crop.

8 Bitwise XOR

$X_i = A_i \oplus B_i$. Unequal input bits produce one.

8.1 Functional circuit diagram



This is a functional port diagram, not a synthesized gate-level schematic.

8.2 Verilog implementation

Algorithm 9: Bitwise XOR: module

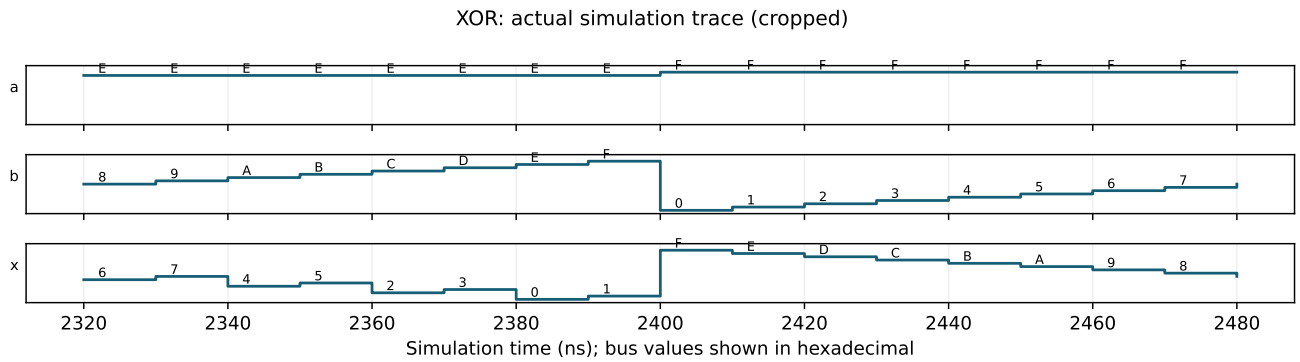
```
‘timescale 1ns/1ps
module alu_xor(input wire [3:0] a, input wire [3:0] b, output wire [3:0] x);
  assign x = a ^ b;
endmodule
```

8.3 Independent unit test

Algorithm 10: Bitwise XOR: exhaustive unit test

```
‘timescale 1ns/1ps
module tb_xor;
  reg [3:0] a, b;
  reg cin;
  wire [3:0] x;
  integer ia, ib, ic, k, expected, remainder, distance, fill, overflow, count;
  alu_xor dut(.a(a), .b(b), .x(x));
  initial begin
    $dumpfile("build/xor.vcd");
    $dumpvars(0, dut);
    count=0;
    for (ia=0; ia<16; ia=ia+1)
      for (ib=0; ib<16; ib=ib+1)
        for (ic=0; ic<1; ic=ic+1) begin
          a=ia; b=ib; cin=ic; #10;
          expected = 0;
          for (k=0; k<4; k=k+1)
            expected = expected | (((6 >> (2*((ia>>k)&1)+((ib>>k)&1))) & 1) << k);
          if (x !== expected[3:0]) $fatal(1,"xor a=%d b=%d",ia,ib);
          count=count+1;
        end
    $display("PASS xor: %0d cases",count);
    $finish;
  end
endmodule
```

8.4 Observed waveform



The crop sweeps B while A=E. Each displayed output follows the per-bit truth table; the testbench checks the full input space beyond this crop.

9 Bitwise XNOR

$X_i = \neg(A_i \oplus B_i)$. Equal input bits produce one.

9.1 Functional circuit diagram



This is a functional port diagram, not a synthesized gate-level schematic.

9.2 Verilog implementation

Algorithm 11: Bitwise XNOR: module

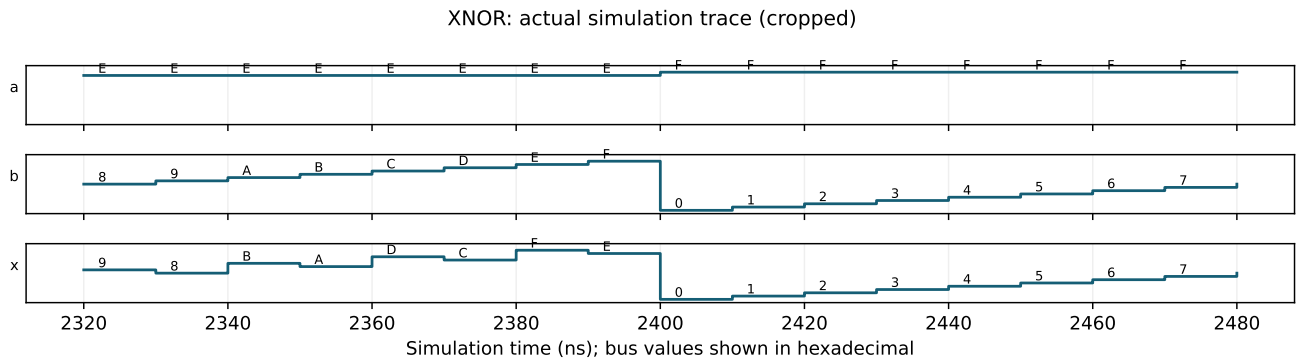
```
‘timescale 1ns/1ps
module alu_xnor(input wire [3:0] a, input wire [3:0] b, output wire [3:0] x);
  assign x = ~(a ^ b);
endmodule
```

9.3 Independent unit test

Algorithm 12: Bitwise XNOR: exhaustive unit test

```
‘timescale 1ns/1ps
module tb_xnor;
  reg [3:0] a, b;
  reg cin;
  wire [3:0] x;
  integer ia, ib, ic, k, expected, remainder, distance, fill, overflow, count;
  alu_xnor dut(.a(a), .b(b), .x(x));
  initial begin
    $dumpfile("build/xnor.vcd");
    $dumpvars(0, dut);
    count=0;
    for (ia=0; ia<16; ia=ia+1)
      for (ib=0; ib<16; ib=ib+1)
        for (ic=0; ic<1; ic=ic+1) begin
          a=ia; b=ib; cin=ic; #10;
          expected = 0;
          for (k=0; k<4; k=k+1)
            expected = expected | (((9 >> (2*((ia>>k)&1)+((ib>>k)&1))) & 1) << k);
          if (x !== expected[3:0]) $fatal(1,"xnor a=%d b=%d",ia,ib);
          count=count+1;
        end
    $display("PASS xnor: %0d cases",count);
    $finish;
  end
endmodule
```

9.4 Observed waveform

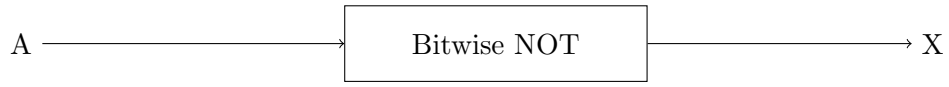


The crop sweeps B while A=E. Each displayed output follows the per-bit truth table; the testbench checks the full input space beyond this crop.

10 Bitwise NOT

$X_i = \neg A_i$. This is a unary operation; there is no B port.

10.1 Functional circuit diagram



This is a functional port diagram, not a synthesized gate-level schematic.

10.2 Verilog implementation

Algorithm 13: Bitwise NOT: module

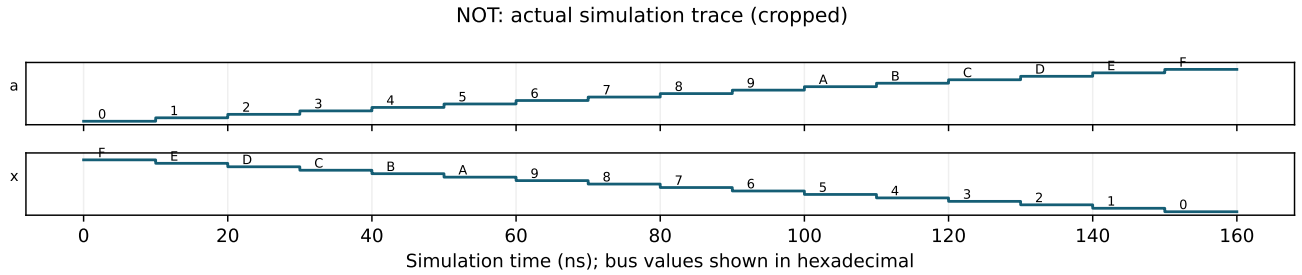
```
`timescale 1ns/1ps
module alu_not(input wire [3:0] a, output wire [3:0] x);
  assign x = ~a;
endmodule
```

10.3 Independent unit test

Algorithm 14: Bitwise NOT: exhaustive unit test

```
`timescale 1ns/1ps
module tb_not;
  reg [3:0] a, b;
  reg cin;
  wire [3:0] x;
  integer ia, ib, ic, k, expected, remainder, distance, fill, overflow, count;
  alu_not dut(.a(a), .x(x));
  initial begin
    $dumpfile("build/not.vcd");
    $dumpvars(0, dut);
    count=0;
    for (ia=0; ia<16; ia=ia+1)
      for (ib=0; ib<1; ib=ib+1)
        for (ic=0; ic<1; ic=ic+1) begin
          a=ia; b=ib; cin=ic; #10;
          expected = 0;
          for (k=0; k<4; k=k+1)
            expected = expected | (((3 >> (2*((ia>>k)&1)+((ib>>k)&1))) & 1) << k);
          if (x !== expected[3:0]) $fatal(1,"not a=%d b=%d",ia,ib);
          count=count+1;
        end
    $display("PASS not: %0d cases",count);
    $finish;
  end
endmodule
```

10.4 Observed waveform



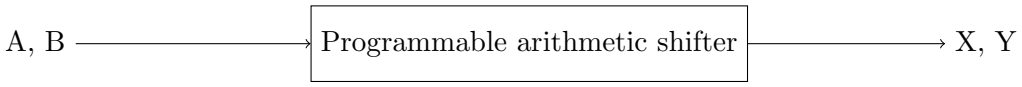
The trace sweeps every input from hexadecimal 0 through F; X descends from F to 0.

11 Programmable arithmetic shifter

The handout defines a programmable fill bit, rather than the automatic sign extension of Verilog's arithmetic-right-shift operator. B[3] selects right (1) or left (0), B[2:1] selects distance 0–3, and B[0] supplies the fill bit. The combinational loop performs up to three one-bit shifts, initializing X to A and Y to zero. Each step captures the departing X bit in Y before replacing it with the fill bit. Left-shift overflow is right-aligned; right-shift overflow is left-aligned. Zero distance preserves A and clears Y. Both assignment examples are covered by exhaustive tests.

For distance s and fill f, left shift gives $X = (A2^s + f(2^s - 1)) \bmod 16$, $Y = \lfloor A/2^{4-s} \rfloor$. Right shift gives $X = \lfloor A/2^s \rfloor + f(16 - 2^{4-s})$, $Y = (A \bmod 2^s)2^{4-s}$. These formulas are also the independent test oracle.

11.1 Functional circuit diagram



This is a functional port diagram, not a synthesized gate-level schematic.

11.2 Verilog implementation

Algorithm 15: Programmable arithmetic shifter: module

```
`timescale 1ns/1ps
// b[3]: direction, b[2:1]: distance, b[0]: fill.
// y collects shifted-out bits, initially zero, in their shift positions.
module alu_shift(input wire [3:0] a, b,
  output reg [3:0] x, y);
  integer i;
  always @* begin
    x = a;
    y = 4'b0000;
    for (i = 0; i < 3; i = i + 1) begin
      if (i < b[2:1]) begin
        if (b[3]) begin
          y = {x[0], y[3:1]};
          x = {b[0], x[3:1]};
        end else begin
          y = {y[2:0], x[3]};
          x = {x[2:0], b[0]};
        end
      end
    end
  end
endmodule
```

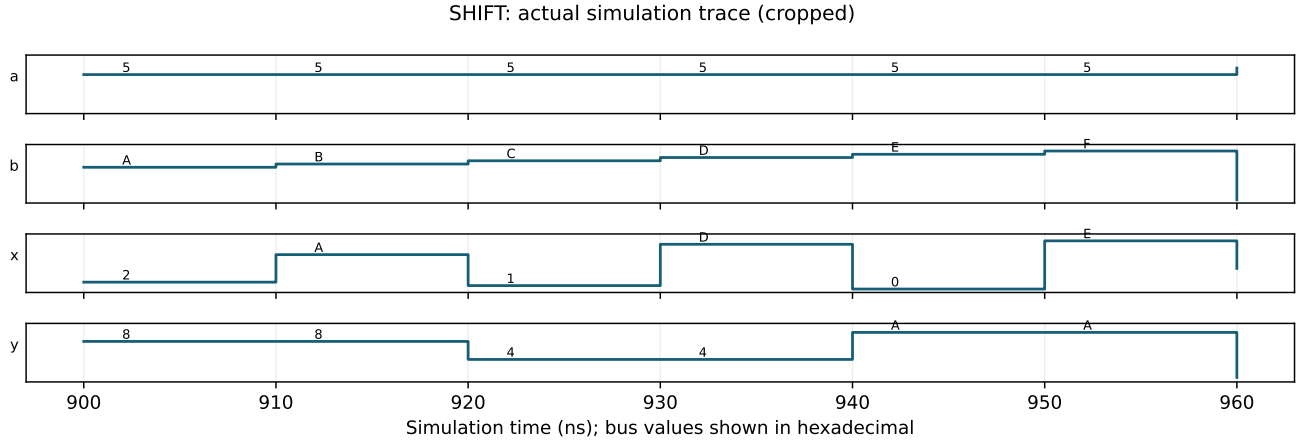
11.3 Independent unit test

Algorithm 16: Programmable arithmetic shifter: exhaustive unit test

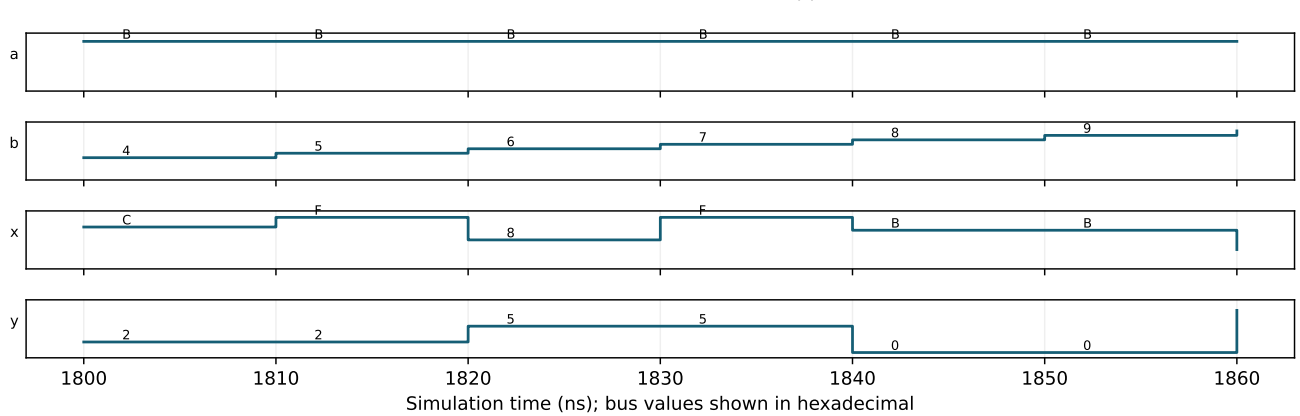
```
`timescale 1ns/1ps
module tb_shift;
  reg [3:0] a, b;
  reg cin;
  wire [3:0] x; wire [3:0] y;
  integer ia, ib, ic, k, expected, remainder, distance, fill, overflow, count;
  alu_shift dut(.a(a), .b(b), .x(x), .y(y));
  initial begin
    $dumpfile("build/shift.vcd");
    $dumpvars(0, dut);
```

```
    count=0;
    for (ia=0; ia<16; ia=ia+1)
      for (ib=0; ib<16; ib=ib+1)
        for (ic=0; ic<1; ic=ic+1) begin
          a=ia; b=ib; cin=ic; #10;
          distance=(ib>>1)&3; fill=ib&1;
          if (ib < 8) begin
            expected=((ia*(1<<distance)) + fill*((1<<distance)-1)) & 15;
            overflow=ia>>(4-distance);
          end else begin
            expected=(ia>>distance) + fill*(16-(1<<(4-distance)));
            overflow=(ia & ((1<<distance)-1)) << (4-distance);
          end
          if (x !== expected[3:0] || y !== overflow[3:0])
            $fatal(1,"shift a=%b b=%b x=%b y=%b",a,b,x,y);
          count=count+1;
        end
    $display("PASS shift: %0d cases",count);
    $finish;
  end
endmodule
```

11.4 Observed waveform



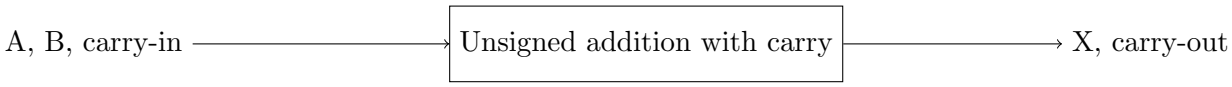
At 930 ns, A=5 and B=D give X=D and Y=4, matching the right-shift example. The second crop shows A=B (hexadecimal eleven), B=6, X=8, Y=5 at 1820 ns, matching the left-shift example.



12 Unsigned addition with carry

A five-bit sum preserves the carry: $T = A + B + C_{in}$, $X = T \bmod 16$, $C_{out} = \lfloor T/16 \rfloor$. Explicit zero extension prevents premature four-bit truncation. For example, $15 + 15 + 1 = 31$ produces X=15 and carry=1. All 512 input combinations are checked.

12.1 Functional circuit diagram



This is a functional port diagram, not a synthesized gate-level schematic.

12.2 Verilog implementation

Algorithm 17: Unsigned addition with carry: module

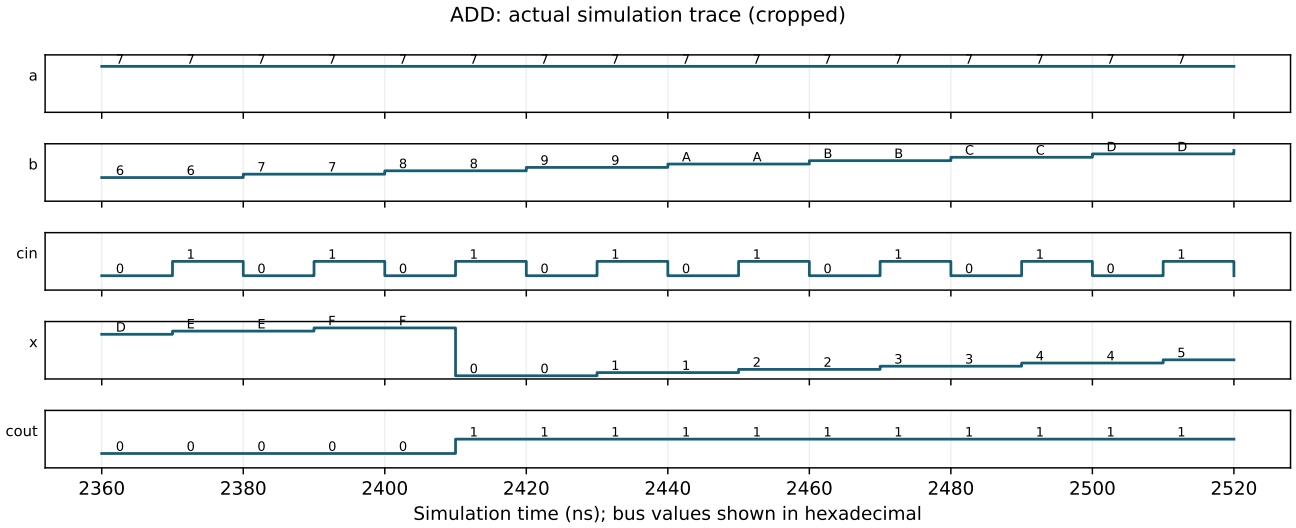
```
‘timescale 1ns/1ps
module alu_add(input wire [3:0] a, b, input wire cin,
  output wire [3:0] x, output wire cout);
  assign {cout, x} = {1’b0,a} + {1’b0,b} + {4’b0,cin};
endmodule
```

12.3 Independent unit test

Algorithm 18: Unsigned addition with carry: exhaustive unit test

```
‘timescale 1ns/1ps
module tb_add;
  reg [3:0] a, b;
  reg cin;
  wire [3:0] x; wire cout;
  integer ia, ib, ic, k, expected, remainder, distance, fill, overflow, count;
  alu_add dut(.a(a), .b(b), .x(x), .cin(cin), .cout(cout));
  initial begin
    $dumpfile("build/add.vcd");
    $dumpvars(0, dut);
    count=0;
    for (ia=0; ia<16; ia=ia+1)
      for (ib=0; ib<16; ib=ib+1)
        for (ic=0; ic<2; ic=ic+1) begin
          a=ia; b=ib; cin=ic; #10;
          expected = ia + ib + ic;
          if (x !== (expected & 15) || cout !== (expected >= 16))
            $fatal(1,"add a=%d b=%d cin=%d",ia,ib,ic);
          count=count+1;
        end
    $display("PASS add: %0d cases",count);
    $finish;
  end
endmodule
```

12.4 Observed waveform

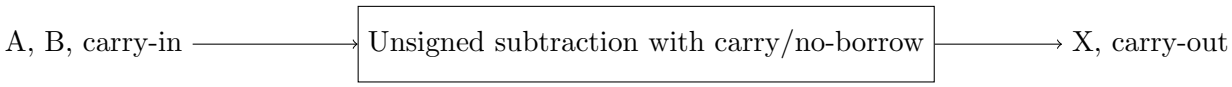


The crop holds A=7 while B crosses 8: carry-out rises as the five-bit sum reaches sixteen; adjacent samples exercise both carry inputs.

13 Unsigned subtraction with carry/no-borrow

The two's-complement adder implements $T = A + \overline{B} + C_{in}$. Consequently, $X = (A - B - (1 - C_{in})) \bmod 16$. For ordinary subtraction use $C_{in} = 1$; use zero to subtract an incoming borrow. $C_{out} = 1$ means no unsigned borrow; borrow-out is its complement. For example, $0 - 1$ with carry-in 1 gives X=15 and carry-out 0. Chaining the carry directly supports multi-nibble subtraction. The bench uses signed integer subtraction and a comparison, not the complemented-adder expression.

13.1 Functional circuit diagram



This is a functional port diagram, not a synthesized gate-level schematic.

13.2 Verilog implementation

Algorithm 19: Unsigned subtraction with carry/no-borrow: module

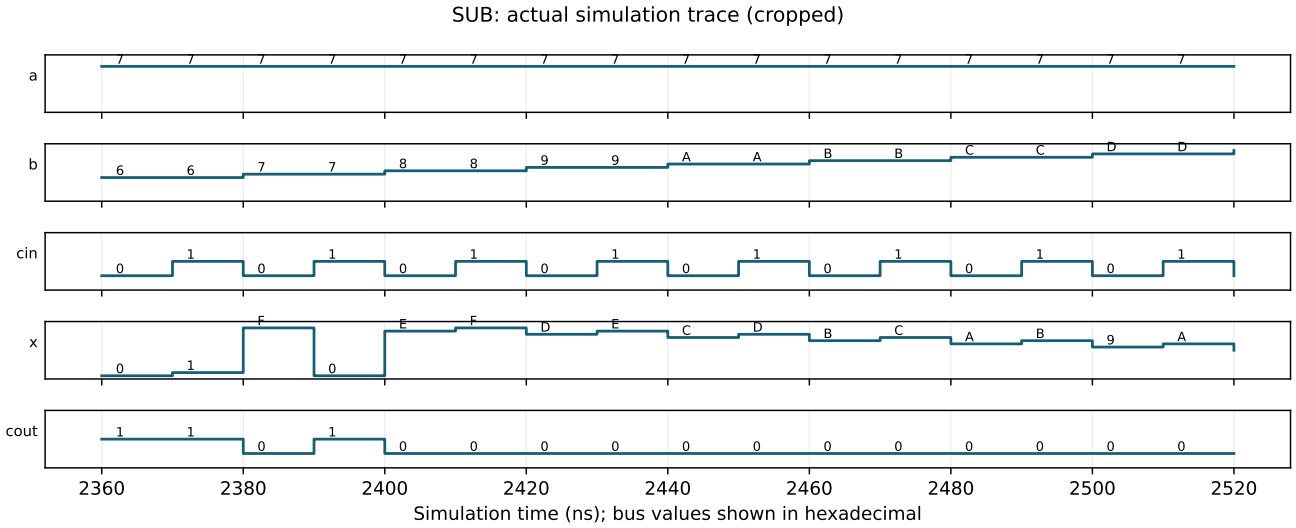
```
`timescale 1ns/1ps
// cin is carry/no-borrow input: x = a - b - (1-cin).
module alu_sub(input wire [3:0] a, b, input wire cin,
  output wire [3:0] x, output wire cout);
  assign {cout, x} = {1'b0,a} + {1'b0,~b} + {4'b0,cin};
endmodule
```

13.3 Independent unit test

Algorithm 20: Unsigned subtraction with carry/no-borrow: exhaustive unit test

```
`timescale 1ns/1ps
module tb_sub;
  reg [3:0] a, b;
  reg cin;
  wire [3:0] x; wire cout;
  integer ia, ib, ic, k, expected, remainder, distance, fill, overflow, count;
  alu_sub dut(.a(a), .b(b), .x(x), .cin(cin), .cout(cout));
  initial begin
    $dumpfile("build/sub.vcd");
    $dumpvars(0, dut);
    count=0;
    for (ia=0; ia<16; ia=ia+1)
      for (ib=0; ib<16; ib=ib+1)
        for (ic=0; ic<2; ic=ic+1) begin
          a=ia; b=ib; cin=ic; #10;
          expected = ia - ib - (1-ic);
          if (x !== (expected & 15) || cout !== (expected >= 0))
            $fatal(1,"sub a=%d b=%d cin=%d",ia,ib,ic);
          count=count+1;
        end
    $display("PASS sub: %0d cases",count);
    $finish;
  end
endmodule
```

13.4 Observed waveform

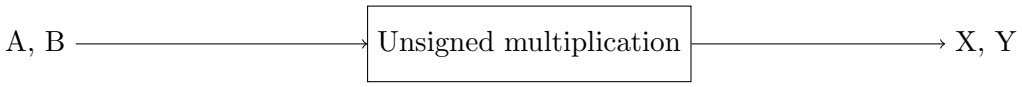


The crop holds A=7 while B crosses 7: carry-out distinguishes no borrow from borrow; alternating carry-in values also exercise incoming borrow.

14 Unsigned multiplication

The eight-bit product is split into low nibble X and high nibble Y, so $16Y + X = AB$. For example, $15 \times 15 = 225$ gives Y=14 and X=1. Both operands are explicitly extended. The test oracle uses repeated addition, independently of the multiplication operator.

14.1 Functional circuit diagram



This is a functional port diagram, not a synthesized gate-level schematic.

14.2 Verilog implementation

Algorithm 21: Unsigned multiplication: module

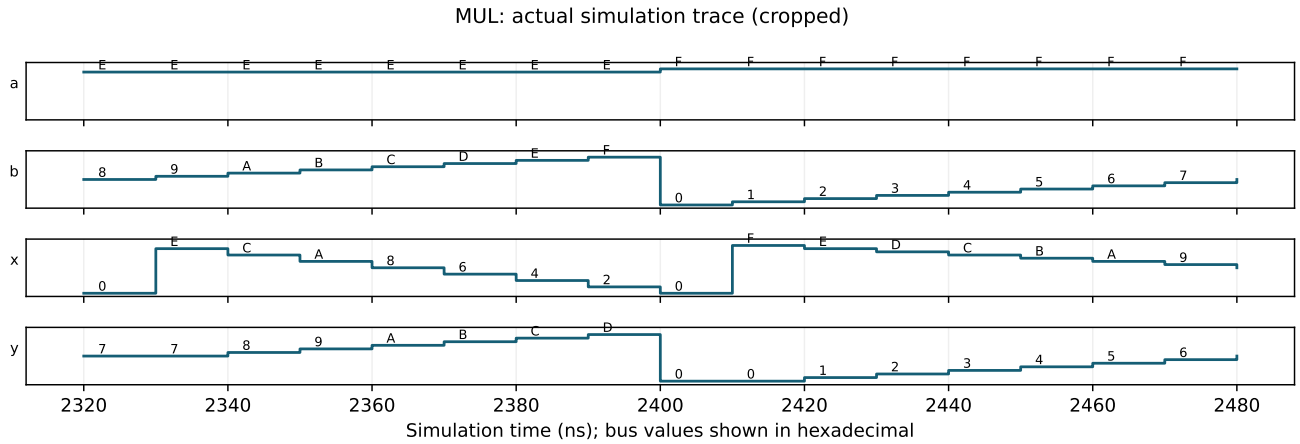
```
‘timescale 1ns/1ps
module alu_mul(input wire [3:0] a, b,
  output wire [3:0] x, y);
  wire [7:0] product;
  assign product = {4’b0,a} * {4’b0,b};
  assign {y,x} = product;
endmodule
```

14.3 Independent unit test

Algorithm 22: Unsigned multiplication: exhaustive unit test

```
‘timescale 1ns/1ps
module tb_mul;
  reg [3:0] a, b;
  reg cin;
  wire [3:0] x; wire [3:0] y;
  integer ia, ib, ic, k, expected, remainder, distance, fill, overflow, count;
  alu_mul dut(.a(a), .b(b), .x(x), .y(y));
  initial begin
    $dumpfile("build/mul.vcd");
    $dumpvars(0, dut);
    count=0;
    for (ia=0; ia<16; ia=ia+1)
      for (ib=0; ib<16; ib=ib+1)
        for (ic=0; ic<1; ic=ic+1) begin
          a=ia; b=ib; cin=ic; #10;
          expected = 0;
          for(k=0;k<ib;k=k+1) expected = expected + ia;
          if ({y,x} !== expected[7:0]) $fatal(1,"mul a=%d b=%d",ia,ib);
          count=count+1;
        end
    $display("PASS mul: %0d cases",count);
    $finish;
  end
endmodule
```

14.4 Observed waveform

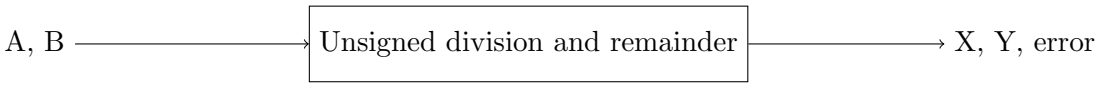


With A=E, increasing B changes both product nibbles. For B=F the product is D2, so Y=D and X=2.

15 Unsigned division and remainder

For nonzero B, X is the integer quotient and Y is the remainder: $A = BX + Y$ and $0 \leq Y < B$. The test oracle uses repeated subtraction. All 240 valid pairs and all 16 zero-divisor pairs are exercised. For B=0, the explicit divide_by_zero output is asserted, X=15, and Y=A. These deterministic sentinel values are not a mathematical quotient and remainder and must be interpreted with the error flag. The handout does not prescribe zero-divisor behavior; this is a documented interface choice.

15.1 Functional circuit diagram



This is a functional port diagram, not a synthesized gate-level schematic.

15.2 Verilog implementation

Algorithm 23: Unsigned division and remainder: module

```
`timescale 1ns/1ps
// Unsigned division. Invalid divisor is explicit, never silently valid.
module alu_div(input wire [3:0] a, b,
  output wire [3:0] x, y, output wire divide_by_zero);
  assign divide_by_zero = (b == 0);
  assign x = divide_by_zero ? 4'hf : a / b;
  assign y = divide_by_zero ? a : a % b;
endmodule
```

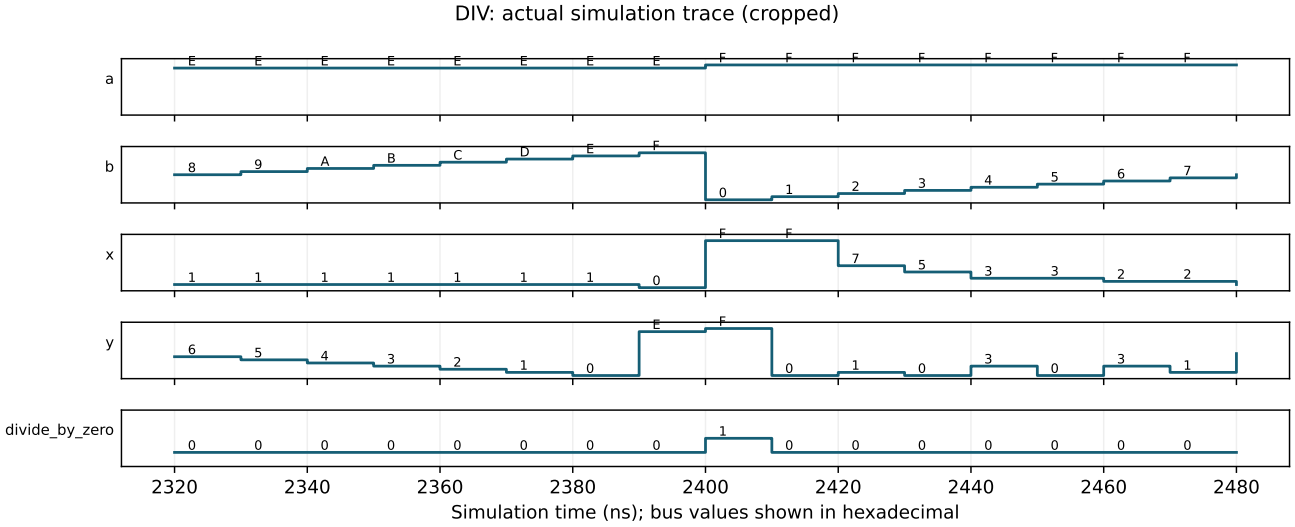
15.3 Independent unit test

Algorithm 24: Unsigned division and remainder: exhaustive unit test

```
`timescale 1ns/1ps
module tb_div;
  reg [3:0] a, b;
  reg cin;
  wire [3:0] x; wire [3:0] y; wire divide_by_zero;
  integer ia, ib, ic, k, expected, remainder, distance, fill, overflow, count;
  alu_div dut(.a(a), .b(b), .x(x), .y(y), .divide_by_zero(divide_by_zero));
  initial begin
    $dumpfile("build/div.vcd");
    $dumpvars(0, dut);
    count=0;
    for (ia=0; ia<16; ia=ia+1)
      for (ib=0; ib<16; ib=ib+1)
        for (ic=0; ic<1; ic=ic+1) begin
          a=ia; b=ib; cin=ic; #10;
          if (ib == 0) begin
            if (divide_by_zero !== 1'b1 || x !== 4'hf || y !== a)
              $fatal(1,"divide by zero a=%d",ia);
          end else begin
            expected=0; remainder=ia;
            while (remainder >= ib) begin
              remainder=remainder-ib; expected=expected+1;
            end
            if (divide_by_zero !== 1'b0 || x !== expected[3:0] || y !== remainder[3:0])
              $fatal(1,"div a=%d b=%d",ia,ib);
          end
          count=count+1;
        end
    $display("PASS div: %0d cases",count);
    $finish;
```

```
  end
endmodule
```

15.4 Observed waveform



The crop includes A=F, B=0 at 2400 ns: error rises, X=F, Y=F. For nonzero B, X and Y return to quotient and remainder.

16 Reproducing the results

From the repository root, run the commands below. Icarus Verilog, Python 3 with Matplotlib, and a LaTeX installation with listings, graphics, and TikZ are required. The test runner also recognizes the local Icarus installation prepared in this cloud environment. No server or credentials are required.

Algorithm 25: Reproduce simulations and report

```
bash scripts/test.sh
python scripts/render_waveforms.py
python scripts/build_report.py
cd latex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

RTL and unit benches are in the Verilog folder. Raw traces and per-unit test logs are generated in build. The report source and vector waveform figures are in latex. The companion archive contains the PDF, source folders, and scripts. Replace the identity placeholders and rebuild the report before naming the archive GroupName-ProjectStep2.zip. Review the implementation and explanations, and adapt this standalone layout to the course template if required.

17 Conclusion

The twelve separate four-bit functions passed exhaustive functional simulation. The two shift examples agree with the assignment, arithmetic outputs preserve carry, high-product bits, and division remainders, and zero-divisor behavior is explicit. The main design ambiguities were overflow alignment in the shifter and the carry convention for subtraction; these were resolved through the supplied examples and a documented no-borrow interface. Independent reference models and full input enumeration make the results stronger than a few selected demonstrations. The next project step can instantiate these modules beneath an opcode controller without merging their implementations. Physical synthesis and optional wider datapaths remain separate work.